

CODE PROBE

For the VIC-20 with VIC-1211A Super Expander

User Manual

```
.....|JO-  
.....J34  
.....W J  
... . IOE .  
  
M2200220L4  
2222222247M  
  
0000L0L0L4  
4M00L0L47M4  
M  
0222222222220  
4004422247M4  
  
00L000000L0M  
00L4M2M47M00  
0  
4.....  
040000400040  
2222224444M  
000000000000  
...44444444440  
  
: 6 1 0 0 0  
H E L L O   W O R L D :  
  
: ■
```


Table of Contents

1. Introduction	1
Features	1
Version history	1
2. Loading and Starting	3
From Tape	3
What happens at startup	3
Recommended invocation from BASIC	3
3. The Monitor Prompt	5
4. Command Reference	7
4.1 A - Alter Mode	7
4.2 D - Hex Dump	8
4.3 R - Display Registers	9
4.4 G - Execute Program	10
4.5 S - Save File	11
4.6 L - Load File	13
4.7 CLS - Clear Screen	14
4.8 EXIT - Exit to BASIC	14
5. Error Messages	17
6. The G Command and Captured Registers	19
How execution works	19
Using G as a breakpoint	19
What if the routine never returns?	19
7. Tutorials	21
7.1 Inspecting Memory	21
7.2 Entering a Machine-Language Program	21
7.3 "Hello World" Program	22

7.4 "Hello World" with a BASIC Stub	23
7.5 Saving and Reloading a Program (PRG)	27
7.6 Saving Raw Data (SEQ)	27
7.7 Inspecting Registers After G	28
7.8 The 3D Cube Round-Trip	29
7.9 Returning to BASIC and Re-entering	30
8. Tips and Tricks	31
8.1 Working faster at the prompt	31
8.2 Saving, loading, and round-tripping	31
8.3 Running ML safely with G	32
8.4 Living with the 22-column screen	32
Appendix A: Memory Map	33
Appendix B: Quick Reference Card	35
Appendix C: Using Code Probe under VICE Emulation	37
C.1 Setting Up the Super Expander via the VICE GUI	37
C.2 Setting Up the Super Expander via the VICE CLI	38

1. Introduction

Code Probe is a minimal machine-language monitor for the Commodore VIC-20. It is a software-based development tool that runs in the 3 KiB expansion RAM of the **VIC-1211A Super Expander** cartridge and produces machine-language programs that run on a stock unexpanded VIC-20 with no cartridge.

The design of Code Probe was inspired by the DOS **DEBUG** utility, and presents a similar terminal-style user interface and commands. All numeric input is hexadecimal: addresses are 4 digits, byte values are 2 digits.

Features

- **Memory inspection** - hex dump with PETSCII character display.
- **Memory editing** - interactive alter mode with cursor navigation and auto-advance.
- **Program execution** - run machine-language programs via JSR / RTS, with post-execution register and flag capture.
- **Tape I/O** - save and load PRG and SEQ files to/from datasette.
- **Register display** - view A, X, Y, and P (with expanded flag bits) captured at the moment the most recent **G** command returned.
- **Screen control** - clear the display with a single command.
- **Exit to BASIC** - return to BASIC's **READY.** prompt; re-enter Code Probe with **SYS 1152**.

Version history

Year	Version	Host	Description
1987	1.0	VIC-20	Hand-assembled and poked into RAM with a BASIC machine language loader.
1988	2.0	C64	Port to C64, with expanded feature set and C64 specific updates.
2026	1.1	VIC-20	Assembly reconstruction of original version 1.0 for the VIC-20.
2026	2.1	C64	Assembly reconstruction of version 2.0 for the C64.

2. Loading and Starting

Code Probe loads at address **\$0480** (1152 decimal) inside the Super Expander cartridge's 3 KiB RAM. The cartridge **must** be inserted at boot; without it, there is no RAM at **\$0480** for Code Probe to live in.

The full PRG file is 2743 bytes: a 13-byte BASIC stub at **\$0401-\$040D**, a zero-fill gap, the monitor body at **\$0480-\$0EB5**, plus the standard 2-byte PRG header.

From Tape

```
LOAD "CODEPROBE",1,1
RUN
```

The **,1,1** parameter loads the program to the address stored in its PRG header (**\$0401**), rather than to the default BASIC area on the expanded machine. **RUN** invokes the BASIC stub, which executes **SYS 1152** and transfers control to Code Probe. Alternatively, **SYS 1152** typed by hand at the BASIC prompt has the same effect.

What happens at startup

1. The screen border and background are set to black via VIC-I register **\$900F**.
2. The KERNAL text colour at **\$0286** is set to white (**\$01**).
3. The screen is cleared via **CHROUT \$93**.
4. The title banner is displayed:

```
CODE PROBE    (1.1)
ROHIN GOSLING (1987)
```

5. A blank line separates the banner from the first prompt.
6. The monitor prompt loop begins.

There is no BRK handler installation step on the VIC-20; the **G** command uses a JSR trampoline rather than a BRK breakpoint mechanism (see Section 6).

Recommended invocation from BASIC

If you intend to use Code Probe alongside BASIC programs in the same session, the following two-step invocation reserves the user ML buffer at **\$1001-\$1DFF** from BASIC's variable area:

```
POKE 55,0:POKE 56,16:CLR
SYS 1152
```

POKE 55,0:POKE 56,16 lowers BASIC's **MEMTOP** to **\$1000**, which prevents BASIC variables from growing into the user's ML buffer if control later returns to BASIC and a program is run. **CLR** re-applies the change. This is purely a safety measure - when invoked directly from autostart, BASIC is dormant and never touches the ML buffer.

3. The Monitor Prompt

The monitor prompt is a colon followed by a space:



Type a command and press **RETURN** to execute it. After each command completes, a blank line is printed, followed by a new prompt.

All numeric values are entered in **hexadecimal**. Addresses are 4-digit hex values (e.g. **1000**, **0480**, **FFFF**). Byte values are 2-digit hex values (e.g. **00**, **FF**, **A5**). The VIC-20 keyboard produces uppercase PETSCII by default, so hex digits **A-F** are entered as plain unshifted letters.

The line buffer holds 19 characters, which is the largest line that fits on the 22-column screen with the **:** prompt and trailing cursor visible. Typing past 19 characters is silently rejected; the cursor will not advance past column 22.

Command dispatch is **single-character**: Code Probe inspects the first letter of the first token only. **C**, **CLS**, and **CLEAR** all route to the clear-screen command; **E**, **EXIT**, and **END** all route to the exit-to-BASIC command.

4. Command Reference

Command	Verb	Summary
A	Alter	Write bytes to RAM.
D	Dump	Display memory as hex / PETSCII.
R	Register	Display the registers and flags captured by G.
G	Go	Execute an ML routine and capture the post-RTS regs.
S	Save	Save a program to tape.
L	Load	Load a program from tape.
CLS	Clear	Clear the screen and home the cursor.
EXIT	Exit	Return to BASIC via the warm-start vector.

4.1 A - Alter Mode

Syntax: A <address>

Enters Alter mode - an interactive hex-digit editor for writing bytes to consecutive memory locations. The monitor prints a flush-left line prompt showing the current address. The user types hex digits *one at a time* via KERNAL GETIN polling. The cursor auto-inserts a separator space after every completed byte and auto-advances to a new prompt line after five bytes (10 nibbles, the screen-width budget).

Line prompt format (6 columns, flush left):

```
AAAA:
```

Four hex digits, colon, space. The cursor follows.

Keys:

Key	Action
0-9, A-F	Enter a hex digit. After every second digit, a space is auto-inserted to mark the byte boundary.
RETURN	Commit any complete bytes on the current line and open a new prompt at the new address. On an empty line, exits Alter mode.
Cursor Left	Move one hex-digit slot to the left, hopping over the separator space at byte boundaries. Clamped at the first slot of the current line.
Cursor Right	Move one hex-digit slot to the right, hopping spaces. Clamped at the entry frontier (the next-to-be-typed slot).
DELETE	At the front, deletes the most recently entered nibble. When the deleted nibble is a low nibble, the trailing separator space is also erased. Ignored when the cursor has been moved back into already-entered data.
Any other key	Silently ignored.

Auto-commit: after the tenth nibble (fifth byte) is entered on a line, Alter mode commits the bytes to memory, advances the line address by 5, and opens a fresh prompt - without requiring **RETURN**. This keeps every full line within the 22-column screen width.

Exiting Alter mode:

Press **RETURN** on an empty line. A byte-count summary is printed flush left:

```
XXXX (Y)
```

XXXX is the total in hex; **Y** is the decimal count. An odd lone high-nibble at exit time is discarded (only complete bytes are committed).

Example session:

```
: A 1000
1000: 00 1F E0 00 FF
1005: 2A 00 F0 00
1009:
0009 (9)

: D 1000 1008
1000: 00 1F E0 00 ....
1004: FF 2A 00 F0 .*..
1008: 00
0009 (9)
```

The first line auto-commits after the tenth nibble (**F** of **FF**); the second line is committed by **RETURN**; the empty third line exits. Five bytes from line 1 plus four from line 2 totals nine bytes.

4.2 D - Hex Dump

Syntax: **D** <start> [<end>]

Displays memory as a hex dump with a PETSCII sidebar. Each row shows the starting address, four bytes in hex, and four printable characters. Non-printable bytes (outside **\$20-\$7E**) are displayed as a period.

Output format (22 columns exact):

```
AAAA: HH HH HH HH CCCC
```

- **AAAA** - address of the first byte in the row.
- **HH** - hex value of each byte.
- **CCCC** - character representation of the same four bytes.

Rows are flush left - there is no leading indent - so each row fits in exactly 22 columns.

Parameters:

- **<start>** - 4-digit hex address (required).
- **<end>** - 4-digit hex address (optional). If omitted, dumps 64 bytes (16 rows of 4 bytes each).

Behaviour:

- Rows are printed from `<start>` to `<end>` inclusive.
- If `<end>` < `<start>`, prints **ERROR: ILLEGAL VALUE**.
- If the dump would read past `$FFFF`, the operation stops at `$FFFF`.
- A byte-count summary is printed on a final line:

```
XXXX (Y)
```

where **XXXX** is the count in hex (zero-padded to 4 digits) and **Y** is the count in decimal. The summary is flush left, matching the indent of the dump rows above it.

Examples:

```
: D 1000 1010
1000: 00 01 02 03 ....
1004: 04 05 06 07 ....
1008: 08 09 0A 0B ....
100C: 0C 0D 0E 0F ....
1010: 10          .
0011 (17)

: D FFFA
FFFA: 43 FE 48 FF C.H.
FFFE: 72 FC          .
0006 (6)
```

When `<end>` is omitted, exactly 64 bytes (16 rows of 4) are printed starting at `<start>`.

4.3 R - Display Registers

Syntax: **R**

Displays the CPU register and processor-status state captured at the end of the most recent **G** command. Read-only - there is no **R <reg> <val>** write form.

Parameters: None. Extra tokens on the R line are ignored.

Output format (three lines, all 22 columns or less):

```
A:XX X:XX Y:XX P:XX
NV-BDIZC
XXXXXXXX
```

- Line 1 (19 chars): A, X, Y, and P as two hex digits each, space-separated.
- Line 2 (8 chars): processor-status flag legend.
- Line 3 (8 chars): the eight bits of P, MSB on the left, each rendered as **'0'** or **'1'**.

Flag-bit positions (MSB to LSB):

Bit	Flag	Name
7	N	Negative
6	V	Overflow
5	-	(unused, always 1)
4	B	Break
3	D	Decimal
2	I	Interrupt-disable
1	Z	Zero
0	C	Carry

Captured state:

- `cmd_g` stores A, X, Y, and the user's P into the shadow register block immediately after the JSR trampoline's RTS returns and a CLD has been issued. The values are whatever the user's routine left in the CPU at the instant it executed RTS.
- Before any G has run during the current Code Probe session, the shadows are all zero, so a bare R right after SYS 1152 shows A:00 X:00 Y:00 P:00 and an all-zero flag row. Not useful, but self-explanatory.

Example:

```
: G 1000      (user routine: leaves A = $42, X = $00, Y = $05, returns)
: R
A:42 X:00 Y:05 P:30
NV-BDIZC
00110000
: █
```

The 30 in the P: value decodes as bits 0011 0000 - bit 4 (B) and bit 5 (the unused "one" bit pushed by PHP) are the only bits set. This is the typical "no flags raised" P value a clean routine leaves behind.

4.4 G - Execute Program

Syntax: G <address>

Transfers control to the machine-language routine at <address> via JSR (through an indirect trampoline). The user's routine returns to Code Probe via RTS.

Parameters:

- <address> - 4-digit hex start address of the user's routine.

Semantics:

- Execution enters via JSR, so the user's routine **must** terminate with RTS to return to Code Probe.

- The CPU registers (A, X, Y, P) are **not** loaded from a saved state before the jump. The user's routine sees whatever register state the call chain produced and is free to modify any register.
- Immediately after the user's RTS, Code Probe issues **CLD** (so a user routine that left the decimal-mode flag set does not corrupt the monitor's own arithmetic), then captures A, X, Y, and the user's P (preserved from a **PHP** taken before the **CLD**) into shadow registers visible via the **R** command.
- The stack pointer is shared with Code Probe; deep user routines must preserve it or reset it before returning.
- No **BRK** handler is installed. If the user's routine fails to **RTS** (e.g. an infinite loop or a jump to BASIC's warm start), control does not return to Code Probe; the user must press **RUN/STOP + RESTORE** or cold-reset the machine.

Example:

```
: A 1000
1000: A9 41 20 D2 FF
1005: 60
1006:
0006 (6)

: G 1000
A
: █
```

The routine loads PETSCII 'A' into the accumulator, calls KERNAL **CHROUT** (**\$FFD2**), and returns. Code Probe prints the **A**, then issues its prompt.

Self-restart trick: **G 0480** re-enters the monitor at its own entry point. The colour / clear / banner sequence re-runs and the user lands at a fresh **:** prompt without leaving the monitor. Each invocation consumes 2 stack bytes (the pending **JSR** return that the re-entered monitor never pops), so this is best used as a one-off "reset the screen state" shortcut, not in a tight loop.

4.5 S - Save File

Syntax (two-line interaction):

```
S <start> <end> <type>
FILE: <name>
```

The **S** command splits across two lines: the first line names the address range and the file format, the second is an auto-prompted filename entry. Splitting the filename off the command line means long names do not have to fit alongside three other tokens on the 22-column screen.

Parameters:

- **<start>** - 4-digit hex start address.
- **<end>** - 4-digit hex end address, inclusive.
- **<type>** - exactly two hex digits. **00** = SEQ (raw bytes, no header). **01** = PRG (2-byte little-endian load-address header prepended).

- **<name>** - up to 16 characters, uppercase PETSCII, **unquoted**. Empty and over-16 both reject as **ERROR: ILLEGAL VALUE**.

File types:

- **<type> = 01** (PRG): KERNAL **SAVE** (\$FFD8) prepends a 2-byte little-endian load-address header. Load with **LOAD "<name>",1,1** from BASIC; launch with **SYS <decimal-start>**. This is the common case for ML routines that should be re-entered by address after reload.
- **<type> = 00** (SEQ): KERNAL **OPEN** + per-byte **CHROUT** writes the range as raw bytes with no header. The file contains exactly **end - start + 1** bytes. Useful for data payloads that should not be confused with loadable programs, and reloadable only via Code Probe's **L <name> <address>** form.

Output:

The KERNAL emits **PRESS RECORD & PLAY ON TAPE** and waits for the user to operate the datasette controls. Once recording begins, KERNAL prints **SAVING <name>**. When the operation completes, the monitor's **:** prompt returns. No additional save banner is printed by Code Probe itself.

Errors:

- **<end> < <start>** - **ERROR: ILLEGAL VALUE** (before any tape I/O).
- **<type>** not exactly **00** or **01** (wrong width, non-hex digit, or out-of-range byte) - **ERROR: ILLEGAL VALUE**.
- Missing or extra tokens on the S-line - **ERROR: ILLEGAL VALUE**.
- Empty or over-16-character filename - **ERROR: ILLEGAL VALUE**.
- Tape write error, **RUN/STOP** mid-save, or KERNAL returns carry set
- **ERROR: FILE ERROR**. The logical file is closed before the prompt returns.

Example (PRG):

```
: S 1000 1005 01
FILE: PRINT-A
PRESS RECORD & PLAY ON TAPE
SAVING PRINT-A

: █
```

Example (SEQ):

```
: S 2000 200F 00
FILE: DATA
PRESS RECORD & PLAY ON TAPE
SAVING DATA

: █
```

(The **PRESS RECORD & PLAY** and **SAVING** messages are emitted by the KERNAL, not by Code Probe.)

4.6 L - Load File

Syntax:

```
L <filename>           - PRG: use the file's header load address.
L <filename> <address> - SEQ: load raw bytes starting at <address>.
```

The presence or absence of **<address>** selects the mode. There is no explicit type flag because the user's intent is unambiguous: a PRG was saved *with* a header (so no destination is needed on reload), a SEQ was saved *without* one (so the user must supply the destination).

Parameters:

- **<filename>** - up to 16 characters, uppercase PETSCII, **unquoted**.
- **<address>** - optional 4-digit hex destination. When supplied, **L** reads the file as SEQ.

Behaviour:

- PRG mode: **SETLFS(1, 1, 1), SETNAM, LOAD(\$00, ...)**. KERNAL reads the file's PRG header block and restores the bytes to the address stored there. Round-trips files saved with **S <start> <end> 1**.
- SEQ mode: **SETLFS(1, 1, 0), SETNAM, OPEN, CHKIN(1)**, per-byte **CHRIN** loop into memory starting at **<address>**. Stops on the KERNAL status byte going non-zero - bit 6 (**\$40**) means clean EOF; any other non-zero value is a tape error.

Output:

The KERNAL emits its standard tape-load message sequence: **PRESS PLAY ON TAPE, SEARCHING FOR <name>, FOUND <name>, LOADING**. When the operation completes, the monitor's **:** prompt returns.

Errors:

- Missing filename - **ERROR: ILLEGAL VALUE**.
- More than two tokens after **L** - **ERROR: ILLEGAL VALUE**.
- Filename longer than 16 chars - **ERROR: ILLEGAL VALUE**.
- Non-hex **<address>** - **ERROR: ILLEGAL VALUE**.
- Tape error, **RUN/STOP**, KERNAL returns carry set, or ST reports a non-EOF error during SEQ read - **ERROR: FILE ERROR**. The logical file is closed.

Examples:

PRG file, header-based restore:

```
: L CUBE
PRESS PLAY ON TAPE
SEARCHING FOR CUBE
FOUND CUBE
LOADING

: █
```

SEQ file, explicit destination:

```
: L DATA 1010  
PRESS PLAY ON TAPE  
SEARCHING FOR DATA  
FOUND DATA  
LOADING
```

```
: █
```

There is no "force PRG to a custom address" mode. KERNAL **LOAD** with secondary address 1 is reliable for PRG files saved by this monitor; if a PRG file's header is wrong, re-save it instead of trying to override at load time.

4.7 CLS - Clear Screen

Syntax: **CLS** (or any token starting with **C**).

Clears the screen and homes the cursor to column 0 of line 0. Emits PETSCII **\$93** via **CHROUT** - the standard KERNAL clear-screen control code. A convenience for wiping a crowded display between experiments, since the 22-column screen fills up quickly.

Parameters: None. Extra tokens are ignored.

Behaviour:

- One **CHROUT \$93** call, no other side effects.
- Cursor blink state is unchanged; the monitor's next **:** prompt renders on the freshly cleared top line.

Example:

```
: D 1000 103F  
(many rows of output filling the screen)  
  
: CLS█
```

RETURN

```
: █
```

4.8 EXIT - Exit to BASIC

Syntax: **EXIT** (or any token starting with **E**).

Returns to BASIC's **READY.** prompt via the BASIC warm-start vector at **\$C002**. Code Probe stays resident in the Super Expander's 3 KiB RAM; re-entry with **SYS 1152** resumes at a fresh monitor prompt.

Parameters: None. Extra tokens are ignored.

Behaviour:

- **JMP (\$C002)** jumps through the BASIC warm-start vector. BASIC's warm-start routine resets its own stack pointer and input state, so Code Probe does not need to tidy up beforehand.

- No BRK-vector restoration is required. The **G** command uses a JSR trampoline rather than a BRK handler, so no interrupt vector has been touched that would need to be put back.
- Re-entry via **SYS 1152** re-runs the colour / clear / banner sequence and re-initialises all input buffers, the nibble buffer, the shadow registers, and the filename buffer. The user's ML payload at **\$1001-\$1DFF** is **not** disturbed.

Example:

```
: EXIT  
  
READY.
```

To return to Code Probe:

```
SYS 1152
```

A cold reset of the VIC-20 does zero the Super Expander's RAM, which loses Code Probe itself; after a cold reset the PRG must be reloaded before another **SYS 1152** will find anything to run.

5. Error Messages

Code Probe defines three error messages. All errors terminate the current command and return the monitor to the prompt.

Message	Cause
ERROR: UNKNOWN COMMAND	The first letter of the command line is not a recognised dispatch letter.
ERROR: ILLEGAL VALUE	A non-hex character was entered, a hex value has the wrong width for the target field, an end-address is less than its start, a filename is empty or over 16 characters, or the token count on a command line is wrong.
ERROR: FILE ERROR	A tape LOAD or SAVE operation failed (read error, write error, tape not present, or the user pressed RUN/STOP).

There is no **ERROR: RAM OVERFLOW**; the **D** command silently stops at **\$FFFF** if the dump range would wrap, and there are no fill / transfer commands to need such a check.

6. The G Command and Captured Registers

Code Probe does not install a **BRK** interrupt handler. The **G** command enters a user routine via a **JSR** through a trampoline, and the user routine returns control by executing **RTS**. The CPU registers are not pre-loaded with a chosen state before the jump - the routine sees whatever register values the call chain produced.

How execution works

1. **cmd_g** parses the address and patches it into a JSR-through- trampoline operand. The trampoline lives in RAM and is essentially:

```
jsr_target:  jmp $0000      ; operand is patched in
```

2. **JSR jsr_target** is executed, which jumps to the user's routine.
3. The user's routine runs with whatever register state the call chain produced. It is responsible for terminating with **RTS**.
4. On **RTS**, control returns immediately to the post-**JSR** instruction in **cmd_g**. The next thing **cmd_g** does is:
 - **PHP** - push the user's processor-status flags onto the stack so the **D** flag (and everything else the user routine left in P) is preserved.
 - **CLD** - clear decimal mode on the live CPU. This makes the monitor's own arithmetic safe even if the user's routine left decimal mode set; without it, the next **ADC** / **SBC** inside the monitor would silently turn into **BCD** arithmetic and the monitor would start mis-parsing every command.
 - Capture A, X, Y, and the popped user P into shadow bytes.
5. The monitor returns to its main prompt loop; **R** will display the captured shadow values.

Using G as a breakpoint

There is no breakpoint mechanism. To inspect register state at an intermediate point in a long routine, split the routine: arrange for it to **RTS** at the point of interest, run with **G**, inspect with **R**, then patch the **RTS** back to the original opcode and continue.

Alternatively, write the routine to **JSR** to a small "stop here" helper that simply returns - but that helper has to be reached by the user's own **JSR**, not Code Probe's, so the captured state would reflect the helper's call site rather than the helper's body.

What if the routine never returns?

If the user's routine enters an infinite loop or jumps to BASIC's warm start without ever executing **RTS**, control does not return to Code Probe. The remedies are:

- **RUN/STOP + RESTORE** - the VIC-20's NMI handler restores the default I/O vectors and returns to BASIC's **READY.** prompt. Code Probe stays resident; re-enter with **SYS 1152**.
- **Cold reset** - last resort; loses Code Probe and any unsaved work.

7. Tutorials

7.1 Inspecting Memory

To examine the contents of memory at any address range, use the **D** command:

```
: D C000 C00F
C000: 78 E3 67 E4 +.|.
C004: 43 42 4D 42 CBMB
C008: 41 53 49 43 ASIC
C00C: 30 C8 41 C7 0.A.
0010 (16)
```

This displays 16 bytes starting at **\$C000**, the entry point of the VIC-20's BASIC ROM. The four-character sidebar on the right shows each byte's on-screen glyph if its value falls in the printable PETSCII range **\$20-\$7E**, and a period otherwise. The first four bytes are two little-endian 16-bit restart vectors: **\$78 \$E3** is the cold-start vector (**\$E378**) and **\$67 \$E4** is the warm-start vector (**\$E467**, the same target that the **EXIT** command jumps through). Their low bytes **\$78** and **\$67** happen to fall in the PETSCII graphics half of the printable range, so they render as graphics characters (**+** and **|**) in the sidebar. Bytes **\$C004-\$C00B** spell the ASCII string **CBMBASIC**, the firmware's signature embedded directly in the ROM. The trailing **0010 (16)** line is the byte-count summary in hex and decimal.

Try **D 8000 800F** to peek at the first 16 bytes of character ROM, or **D 1E00 1E0F** to see what the **:** prompt and your most recent commands look like in screen RAM.

7.2 Entering a Machine-Language Program

Use the **A** command to enter bytes into RAM. The simplest useful example is a 6-byte routine that prints a single character:

```
: A 1000
1000: A9 41 20 D2 FF
1005: 60
1006:
0006 (6)
```

The first line auto-commits after the fifth byte (**FF**). The second line contains a single **60** (the **RTS** opcode); press **RETURN** to commit it, then **RETURN** again on the empty **1006:** line to exit Alter mode.

The program decodes as:

Hex	Assembly	Description
A9 41	LDA #\$41	Load PETSCII 'A'.
20 D2 FF	JSR \$FFD2	Call KERNAL CHROUT .
60	RTS	Return to Code Probe.

Verify with **D**:

```
: D 1000 1005
1000: A9 41 20 D2 .A .
1004: FF 60      .
0006 (6)
```

Execute with **G**:

```
: G 1000
A
: █
```

7.3 "Hello World" Program

A meatier example: a 37-byte program that prints **HELLO WORLD!** between two blank lines and returns to the monitor.

Program listing:

Address	Hex	Assembly	Description
\$1000	A9 0D	LDA #\$0D	Carriage return.
\$1002	20 D2 FF	JSR \$FFD2	Print blank line.
\$1005	A2 00	LDX #\$00	Index = 0.
\$1007	BD 18 10	LDA \$1018,X	Fetch next char.
\$100A	F0 06	BEQ \$1012	NUL terminator - done.
\$100C	20 D2 FF	JSR \$FFD2	Print char.
\$100F	E8	INX	Advance.
\$1010	D0 F5	BNE \$1007	Loop.
\$1012	A9 0D	LDA #\$0D	Carriage return.
\$1014	20 D2 FF	JSR \$FFD2	Print blank line.
\$1017	60	RTS	Return.
\$1018	48 45 4C 4C	-	"HELL"
\$101C	4F 20 57 4F	-	"O WO"
\$1020	52 4C 44 21	-	"RLD!"
\$1024	00	-	NUL terminator.

Step 1 - enter the program.

With auto-commit firing every five bytes, the Alter-mode transcript is exactly:

```

: A 1000
1000: A9 0D 20 D2 FF
1005: A2 00 BD 18 10
100A: F0 06 20 D2 FF
100F: E8 D0 F5 A9 0D
1014: 20 D2 FF 60 48
1019: 45 4C 4C 4F 20
101E: 57 4F 52 4C 44
1023: 21 00
1025:
0025 (37)

```

The empty **RETURN** on the **1025:** line exits Alter mode.

Step 2 - verify with D.

```

: D 1000 1024
1000: A9 0D 20 D2 . . .
1004: FF A2 00 BD . . .
1008: 18 10 F0 06 . . .
100C: 20 D2 FF E8 . . .
1010: D0 F5 A9 0D . . .
1014: 20 D2 FF 60 . .
1018: 48 45 4C 4C HELL
101C: 4F 20 57 4F O WO
1020: 52 4C 44 21 RLD!
1024: 00 .
0025 (37)

```

The **HELL** / **O WO** / **RLD!** runs are visible in the PETSCII sidebar at **\$1018-\$1023**.

Step 3 - run with G.

```

: G 1000

HELLO WORLD!

: █

```

7.4 "Hello World" with a BASIC Stub

A more complete Hello World. The program switches the screen to black text on a white background with a white border, clears the screen so the new colour scheme takes effect across the entire display, prints **HELLO WORLD!** followed by a blank line and then **PRESS ANY KEY...**, and finally waits for any keypress before returning to the caller.

It also includes a BASIC stub so it can be launched by **RUN** from BASIC's **READY.** prompt as an alternative to **G 100E** from Code Probe. The program lives at the start of the unexpanded VIC-20's BASIC program area (**\$1001**), the same address BASIC's **RUN** expects to find a tokenised program at, so the entire 97-byte payload behaves like a normal BASIC program from the point of view of **LOAD**, **RUN**, and **LIST**.

Layout:

The program is in two parts:

1. A 13-byte BASIC stub at **\$1001-\$100D** containing the tokenised line **10 SYS 4110**. This is the same upstart-stub pattern Code Probe itself uses. **RUN** parses the line and dispatches via the **SYS** jump.
2. The ML body at **\$100E-\$1061** (84 bytes), reachable as **SYS 4110** from BASIC or as **G 100E** from Code Probe.

Program listing:

Address	Hex	Assembly	Description
\$1001	0C 10 0A 00	-	Link to \$100C, line# 10.
\$1005	9E 20	-	SYS token + space.
\$1007	34 31 31 30	-	"4110".
\$100B	00 00 00	-	End of line + end of program.
\$100E	A9 19	LDA #\$19	\$19 = white bg, white border.
\$1010	8D 0F 90	STA \$900F	Set VIC-I bg/border.
\$1013	A9 00	LDA #\$00	Black.
\$1015	8D 86 02	STA \$0286	KERNAL text colour.
\$1018	A9 93	LDA #\$93	CHR\$(147) = clear screen.
\$101A	20 D2 FF	JSR \$FFD2	Clear; new cells take colour.
\$101D	A2 00	LDX #\$00	Index = 0.
\$101F	BD 42 10	LDA \$1042,X	Fetch next char of msg1.
\$1022	F0 06	BEQ \$102A	NUL terminator - done.
\$1024	20 D2 FF	JSR \$FFD2	Print char.
\$1027	E8	INX	Advance.
\$1028	D0 F5	BNE \$101F	Loop.
\$102A	A9 0D	LDA #\$0D	Carriage return.
\$102C	20 D2 FF	JSR \$FFD2	Print blank line.
\$102F	A2 00	LDX #\$00	Index = 0.
\$1031	BD 50 10	LDA \$1050,X	Fetch next char of msg2.
\$1034	F0 06	BEQ \$103C	NUL terminator - done.
\$1036	20 D2 FF	JSR \$FFD2	Print char.
\$1039	E8	INX	Advance.
\$103A	D0 F5	BNE \$1031	Loop.
\$103C	20 E4 FF	JSR \$FFE4	KERNAL GETIN - A=key or 0.
\$103F	F0 FB	BEQ \$103C	Spin while no key.
\$1041	60	RTS	Return to caller.

Address	Hex	Assembly	Description
\$1042	48 45 4C 4C	-	"HELL"
\$1046	4F 20 57 4F	-	"O WO"
\$104A	52 4C 44 21	-	"RLD!"
\$104E	0D 00	-	CR + NUL terminator.
\$1050	50 52 45 53	-	"PRES"
\$1054	53 20 41 4E	-	"S AN"
\$1058	59 20 4B 45	-	"Y KE"
\$105C	59 2E 2E 2E	-	"Y..."
\$1060	0D 00	-	CR + NUL terminator.

The \$19 value written to \$900F decodes as bg=0001 (white, nibble 7-4), bit 3 = 1 (normal mode, not reversed), and border=001 (white, bits 2-0). The KERNAL clear-screen call at \$101A paints the colour-RAM cells with whatever value is at \$0286, which is why the text colour is set immediately before the clear.

Step 1 - enter the program.

```
: A 1001
1001: 0C 10 0A 00 9E
1006: 20 34 31 31 30
100B: 00 00 00 A9 19
1010: 8D 0F 90 A9 00
1015: 8D 86 02 A9 93
101A: 20 D2 FF A2 00
101F: BD 42 10 F0 06
1024: 20 D2 FF E8 D0
1029: F5 A9 0D 20 D2
102E: FF A2 00 BD 50
1033: 10 F0 06 20 D2
1038: FF E8 D0 F5 20
103D: E4 FF F0 FB 60
1042: 48 45 4C 4C 4F
1047: 20 57 4F 52 4C
104C: 44 21 0D 00 50
1051: 52 45 53 53 20
1056: 41 4E 59 20 4B
105B: 45 59 2E 2E 2E
1060: 0D 00
1062:
0061 (97)
```

The first nineteen lines auto-commit after their fifth byte each. 1060: 0D 00 is committed by **RETURN**, and **RETURN** on the empty 1062: prompt exits Alter mode.

Step 2 - run from Code Probe.

```
: G 100E
```

RETURN

```
HELLO WORLD!  
  
PRESS ANY KEY...
```

Press any key...

```
: █
```

The screen turns white, then **HELLO WORLD!**, a blank line, and **PRESS ANY KEY...** print starting at the top in black text. The routine spins on KERNAL **GETIN** until a key is pressed, then **RTS** returns to Code Probe's prompt loop. Because the routine deliberately overwrites the screen colours, Code Probe's next **:** prompt renders black-on-white; to restore the default white-on-black palette, type **EXIT** followed by **SYS 1152**.

Step 3 - save to tape and run from BASIC.

Save the whole 97-byte payload, BASIC stub included, in PRG mode:

```
: S 1001 1061 01  
FILE: HELLO2  
PRESS RECORD & PLAY ON TAPE  
SAVING HELLO2  
  
: █
```

- Power off the VIC-20.
- Remove the VIC-1211A Super Expander cartridge.
- Power the VIC-20 back on in stock unexpanded mode.
- Load "HELLO2" from tape.

```
**** CBM BASIC VERSION 2 ****  
3583 BYTES FREE  
READY.  
  
LOAD "HELLO2", 1, 1  
  
PRESS PLAY ON TAPE  
OK  
  
SEARCHING FOR HELLO2  
FOUND HELLO2  
  
LOADING  
READY.  
  
RUN █
```

RETURN

```
HELLO WORLD!  
  
PRESS ANY KEY...
```

Press any key...

```
HELLO WORLD!

PRESS ANY KEY...

READY.
█
```

RUN parses the BASIC stub, dispatches **SYS 4110**, and the ML routine runs. Pressing any key returns to BASIC's **READY.** prompt rather than to Code Probe. **SYS 4110** typed at the **READY.** prompt has the same effect.

7.5 Saving and Reloading a Program (PRG)

After entering the Hello World program, save it to tape as a PRG file:

```
: S 1000 1024 01
FILE: HELLO
PRESS RECORD & PLAY ON TAPE
SAVING HELLO

: █
```

The 2-byte PRG header records the load address **\$1000**, so the file is loadable by BASIC on a stock VIC-20 with **LOAD "HELLO",1,1** — note however that **RUN** will not work because there is no BASIC stub; launch with **SYS 4096** instead.

To verify the round trip from Code Probe, clear the program area and reload:

```
: A 1000
1000: 00 00 00 00 00
1005: 00 00 00 00 00
      (continue zeroing $1000-$1024 then RETURN twice to exit)

: L HELLO
PRESS PLAY ON TAPE
SEARCHING FOR HELLO
FOUND HELLO
LOADING

: G 1000

HELLO WORLD!

: █
```

7.6 Saving Raw Data (SEQ)

If you want to save a region of memory without a load-address header — for example a screen capture or a data table - use SEQ mode by passing **00** as the type. The reload then needs an explicit destination address:

```
: S 1E00 1FF9 00
FILE: SCREEN
PRESS RECORD & PLAY ON TAPE
SAVING SCREEN
```

```
: █
```

```
: L SCREEN 1E00
PRESS PLAY ON TAPE
SEARCHING FOR SCREEN
FOUND SCREEN
LOADING
```

```
: █
```

The save captures the entire 22 x 23 screen RAM (506 bytes) without any header. The reload places it back at **\$1E00**, restoring the displayed image.

7.7 Inspecting Registers After G

The **R** command shows what the CPU looked like at the instant the most recent **G** command's user routine returned. To see this in action, enter a small routine that loads known values:

```
: A 1100
1100: A9 42 A2 07 A0
1105: 09 60
1107:
0007 (7)
```

The bytes decode as:

Hex	Assembly	Description
A9 42	LDA #\$42	Load \$42 into accumulator.
A2 07	LDX #\$07	Load \$07 into X.
A0 09	LDY #\$09	Load \$09 into Y.
60	RTS	Return.

Run and inspect:

```
: G 1100
: R
A:42 X:07 Y:09 P:30
NV-BDIZC
00110000
```

```
: █
```

The shadow registers persist until the next **G** runs. Run another routine, and they update.

7.8 The 3D Cube Round-Trip

This tutorial walks through the full deployment workflow: author an ML program using Code Probe under the Super Expander, save it to tape, then load and run it on a stock cartridge-less VIC-20.

Prerequisites:

- A VIC-20 with the Super Expander attached and Code Probe running.
- The cube payload resident in user RAM at **\$1001-\$1DD4** (about 3540 bytes). In practice this means either entering it byte-by-byte via one or more **A** sessions, or reloading it from a tape that an earlier authoring session had saved. As a point of interest, the original 3D cube program was authored entirely in Code Probe.

Step 1 - get the cube into RAM at **\$1001**.

If you have a tape with an earlier **S** of the cube in PRG mode, reload it now:

```
: L CUBE
PRESS PLAY ON TAPE
SEARCHING FOR CUBE
FOUND CUBE
LOADING

: █
```

The PRG header restores the bytes to **\$1001** automatically.

Step 2 - verify the bytes.

```
: D 1001 1014
```

The first row should match the cube's BASIC stub bytes (a tokenised **10 SYS 4110**).

Step 3 - save to tape.

Insert a fresh tape into the datasette, then:

```
: S 1001 1DD4 01
FILE: CUBE
PRESS RECORD & PLAY ON TAPE
SAVING CUBE

: █
```

This is a large save, and you can expect it to take noticeably longer than the small-payload examples above.

Step 4 - relaunch on a stock unexpanded VIC-20.

Power the VIC-20 off, remove the Super Expander cartridge, and power back on. The machine now boots as a plain unexpanded VIC-20. Keep the tape from Step 3 in the datasette.

Step 5 - load and run on the stock VIC-20.

At BASIC's **READY.:**

```
LOAD "CUBE",1,1  
SYS 4110
```

The cube animation runs, on a cartridge-less unexpanded VIC-20. This is the whole reason Code Probe exists.

7.9 Returning to BASIC and Re-entering

To leave Code Probe without losing it from memory:

```
: EXIT  
  
READY.
```

Now you are at BASIC's prompt. You can run BASIC programs, type calculator expressions, or do anything BASIC does. Code Probe is still in the Super Expander's RAM at **\$0480**. To go back:

```
SYS 1152
```

Code Probe re-runs its startup sequence - screen colours, clear, banner, prompt - but the user's ML buffer at **\$1001-\$1DFF** is untouched. Anything you entered there with **A**, or loaded with **L**, is still present.

A cold reset (powering the VIC-20 off and on) zeros the Super Expander's RAM, which loses Code Probe. Reload it from tape before the next **SYS 1152**.

8. Tips and Tricks

This section collects the small body of expert-user wisdom that is not obvious from the command syntax alone - shortcuts, workarounds, and rough edges that experienced users tend to learn either by reading the source or by burning themselves once. None of it is required reading; the sections above cover everything you strictly need to drive Code Probe. But the first time one of these tips saves you an hour of head-scratching, you will be glad it was written down.

8.1 Working faster at the prompt

- **Single-character dispatch** - Only the first letter of every command matters. **C**, **CLS**, and **CLEAR** all clear the screen; **E**, **EXIT**, and **END** all return to BASIC. Type whichever form reads naturally to you - on a 22-column screen the short form is almost always preferable.
- **Auto-commit in Alter mode** - After ten hex digits (five bytes) on an Alter row, the monitor commits the line and opens a fresh prompt at the next address automatically; no **RETURN** needed. On long edits you can hold the keys down and just keep typing through the entire region. **RETURN** is only required for partial rows of fewer than five bytes, and to exit Alter mode entirely.
- **G \$0480 as a soft reset** - Re-entering the monitor at its own start address re-runs the banner, screen colours, and clear sequence. It is a cheap one-shot way to reset visible screen state mid-session without exiting to BASIC. Each invocation consumes two stack bytes that are never recovered, so it is intended for occasional use, not a loop.
- **Why your input cuts off at 19 characters** - The line buffer leaves three columns for the **:** prompt and the cursor on the 22-column screen. The **19** is not a configuration choice; it is the largest line that fits. Plan command lines accordingly. Very long Alter regions are easier as multiple **A** invocations than as a single overflowing **A** line.

8.2 Saving, loading, and round-tripping

- **PRG vs SEQ - when to use which** - PRG saves prepend a 2-byte load-address header; reload with **LOAD "name",1,1** and the file goes back to the address it came from, ready to **SYS** directly. SEQ saves are raw bytes with no header; reload requires you to specify the destination address explicitly. Use PRG for self-contained programs you intend to re-run; use SEQ for data payloads or whenever you need to relocate the bytes to a different address on reload.
- **The round-trip workflow** - Author and debug your routine on the Super Expander, save it as PRG with **S**, quit Code Probe, and on a stock cartridge-less VIC-20 run **LOAD "name",1,1** followed by **SYS <start-address>**. The whole point of Code Probe is that the programs it produces do not need it to run; the cartridge is the development environment, not a runtime dependency.
- **Code Probe can save a copy of itself** - The monitor image spans **\$0401-\$0EB5**: the BASIC upstart stub at **\$0401** followed by the monitor body at **\$0480**. **S 0401 0EB5 01** saves the whole thing as a PRG; name it **CODEPROBE** at the **FILE:** prompt. The resulting file reloads on any Super-Expanded VIC-20 with **LOAD "CODEPROBE",1,1** followed by **RUN** - the stub's **SYS 1152** re-enters the monitor. The save range deliberately starts at the stub rather than the monitor body, so **RUN** works without a manual **SYS 1152** after load. Useful for capturing a known-good build before experimenting with a patched copy in RAM.
- **Why the FILE: prompt is on its own line** - The 22-column screen cannot fit **<start> <end> <type> <16-char filename>** on one line, so **S** splits the input across two lines: address range and type on the

first, filename on a second **FILE:** prompt. Filenames are uppercase PETSCII, unquoted, and capped at 16 characters - the standard CBM filename limit.

- **Exit and re-enter for a clean slate** - **EXIT** returns to BASIC's **READY.** prompt; **SYS 1152** re-enters Code Probe with a fresh banner. Anything in your ML buffer at **\$1001-\$1DFF** survives the round trip - this is a useful way to reset the screen and the monitor's internal state without losing work.

8.3 Running ML safely with G

- **Your routine must end in RTS, not RTI or BRK** - Code Probe enters routines via **JSR** through a self-modifying trampoline; only **RTS** returns control cleanly. There is no **BRK** handler installed, and **RTI** will pull garbage off the stack. If your routine wants to terminate early, **JMP** to a tail **RTS** - never **BRK**.
- **Do not leave decimal mode set** - If your routine executes **SED** and forgets to **CLD** before its final **RTS**, every subsequent monitor command will fail with **ERROR: UNKNOWN COMMAND** because the monitor's own arithmetic silently goes BCD. Code Probe issues **CLD** immediately after capturing your processor flags on a clean return, but a routine that never reaches its **RTS** leaves the flag set on the live CPU. If the monitor stops accepting commands after a **G**, this is almost always why - re-enter Code Probe with **EXIT** followed by **SYS 1152** to recover.
- **Inspecting registers mid-routine without breakpoints** - Code Probe has no breakpoint mechanism. To inspect register state at an intermediate point in a routine, temporarily patch an **RTS (\$60)** over the next instruction with **A**, run the routine with **G**, inspect the captured registers with **R**, then restore the original byte and continue. It is crude, but it is the only mechanism available.
- **R is read-only** - There is no write-side **R <register> <value>** form that pre-loads the CPU with a chosen state before the next **G**. To pre-load registers for a routine, set them inside the routine itself with explicit **LDA #...** / **LDX #...** / **LDY #...** instructions.

8.4 Living with the 22-column screen

- **The 22-character wrap quirk** - Writing exactly 22 characters to the VIC-20 screen auto-wraps the cursor as a side effect of the final **CHROUT**. A subsequent explicit carriage return then advances the cursor a *second* time, leaving a spurious blank row between outputs. Code Probe's dump rows are exactly 22 columns wide and intentionally omit the trailing carriage return for this reason. If you write your own screen-output routines, follow the same pattern: skip the **CR** after any row that is exactly 22 characters wide.
- **Why dump rows are flush-left** - Dump rows have no leading indent because the four address digits, six data bytes, and four PETSCII characters add up to exactly 22 columns. Adding even a single space of indent would push the row past the right margin and trigger the wrap quirk above. The eight-bytes-per-row layout common on wider monitors is simply not expressible on a 22-column screen.
- **BASIC's MEMTOP can clobber your ML** - If you **SYS 1152** from BASIC and intend to return to BASIC and keep working there, run **POKE 55,0 : POKE 56,16 : CLR** before invoking Code Probe. This lowers BASIC's **MEMTOP** to **\$1000** and protects the **\$1001-\$1DFF** ML buffer from being overwritten by BASIC variable storage. Without this, a sufficiently large BASIC program or variable allocation can quietly trample bytes you entered with **A** or loaded with **L**.

Appendix A: Memory Map

Code Probe resides at **\$0480-\$0EB5** inside the Super Expander cartridge's 3 KiB expansion RAM. The user's ML buffer is at **\$1001-\$1DFF** in the VIC-20's native RAM, sized to match the unexpanded-VIC-20 BASIC program area so that programs developed under Code Probe deploy unchanged on stock hardware.

\$FFFF	KERNAL ROM
\$E000	
\$DFFF	(open)
\$C000	BASIC ROM
\$AFFF	
\$A000	Super Expander BASIC ext. ROM
\$9400	
	Color RAM
\$9000	VIC-I, VIA1, VIA2 (I/O)
\$8000	
	Character ROM
\$2000	
	(no RAM - BLK1 absent)
\$1FFF	
\$1E00	Screen RAM (22x23)
\$1DFF	
	USER ML BUFFER (\$1001-\$1DFF) (~3583 bytes, free for ML)
\$1001	
\$1000	User-buffer guard byte
\$0FFF	
	(top of Super Expander 3K)
\$0EB6	
	CODE PROBE 1.1 (\$0480-\$0EB5)
\$0480	
	(zero-fill gap)
\$040E	
\$0401	BASIC upstart stub (10 SYS 1152), used by autostart
\$0400	
	KERNAL / BASIC work area
\$0200	
	6502 hardware stack
\$0100	
	Zero page
\$0000	

Safe areas for user programs: **\$1001-\$1DFF** (about 3.5 KiB of contiguous RAM, sized to match the BASIC program area on a stock unexpanded VIC-20). Programs entered here will load at the same address on a cartridge-less machine after **S** to tape and **LOAD "<name>",1,1** from BASIC.

Avoid writing to:

- **\$0400-\$0FFF** - Super Expander RAM (Code Probe lives here).

- **\$1E00-\$1FF9** - screen RAM (writing here puts characters on screen, which is sometimes intentional but usually surprising).
 - **\$9000-\$9FFF** - VIC-I, VIA1, VIA2, and color RAM.
 - **\$A000-\$AFFF** - Super Expander BASIC extension ROM.
 - **\$C000-\$FFFF** - BASIC ROM and KERNAL ROM.
-

Appendix B: Quick Reference Card

Cmd	Syntax	Description
A	A <address>	Enter Alter mode to write hex bytes to RAM. 5 bytes per line auto-commit; RETURN to commit partial; empty RETURN to exit.
D	D <start> [<end>]	Hex dump memory from start to end (inclusive). Default end = start + 64.
R	R	Display registers and processor flags captured by the most recent G command. Read-only.
G	G <address>	Execute ML routine at <address>. User routine must end with RTS. Captures A, X, Y, P after RTS.
S	S <start> <end> <type>	Save range [start, end] to tape; the monitor prompts FILE: <name> on a second line. type=01 -> PRG (with header), type=00 -> SEQ (raw).
L	L <name> or L <name> <address>	Load PRG file (without <address>, uses file's header) or SEQ raw bytes (with <address>).
CLS	CLS	Clear screen and home cursor.
EXIT	EXIT	Return to BASIC (READY. prompt). Re-enter with SYS 1152.

All address values are 4-digit hexadecimal. Byte values are 2-digit hexadecimal. Filenames are unquoted, up to 16 characters of uppercase PETSCII. Tape is the only I/O device (KERNAL device 1).

Appendix C: Using Code Probe under VICE Emulation

The preceding sections of this manual describe Code Probe's behaviour on real VIC-20 hardware with a physical VIC-1211A Super Expander cartridge. The VICE emulator can host the same software stack on a desktop computer, but the cartridge must be configured by hand because VICE does not ship the Super Expander ROM. This section describes the two configuration paths - the GUI-driven settings dialogs in the VICE menu bar, and the equivalent command-line invocation. Once the emulator is configured, the rest of this manual applies unchanged.

The VIC-1211A is two devices on one cartridge: 3 KiB of RAM at **\$0400-\$0FFF**, and a 4 KiB BASIC-extension ROM at **\$A000-\$AFFF**. VICE configures these as separate options, which is why both a "memory" setting and a "cartridge" setting must be supplied.

Both paths assume you have already obtained a Super Expander ROM image. The Zimmers archive distributes a 4098-byte PRG with a **\$A000** load-address header at: Super Expander.prg

> **Note:** There are two versions of the VIC-1211 Super Expander available for download from zimmers.net. They are the VIC1211m, which was a version originally targeting the Japanese market in the 1980s, and the regular VIC1211A, which is labeled **Super Expander.prg** on the zimmers.net download page. For the purpose of running Code Probe in VICE, you will want to download and attach the regular VIC1211A labeled **Super Expander.prg** on the zimmers.net download page.

Save the file locally; it is not redistributable inside the emulator.

C.1 Setting Up the Super Expander via the VICE GUI

1. **Enable Block 0 RAM.** From the menu bar:

Preferences -> Settings -> Machine -> Model -> RAM blocks

Tick **Block 0 (3 KiB at \$0400-\$0FFF)** and untick all other RAM blocks. The Zimmers ROM file does not bring its own RAM, so without this step Code Probe has nowhere to live.

2. **Attach the ROM as a cartridge.** From the menu bar:

File -> Attach cartridge image

In the file picker, change the **Cartridge type** dropdown from **Smart-attach** to **Generic cartridge**, then select the Super Expander ROM PRG. Smart-attach expects a **.crt** header or a filename hint; a plain PRG will fail with **Failed to attach image**. Generic cartridge reads the 2-byte PRG header directly and places the 4 KiB at block 5. If a narrower option such as **4/8 KiB cartridge at \$A000** is offered, prefer it.

3. **Cold-reset the emulator.** **Machine -> Reset -> Hard reset.**

4. **Verify the cartridge is active.** At BASIC's **READY.** prompt:

- The boot banner reports fewer than 6655 free bytes (the extension ROM reserves graphics workspace out of the 3 KiB). If the banner shows exactly 6655, only the RAM took effect — re-attach the cartridge.
- **GRAPHIC 2 : CIRCLE 1,512,512,300,300** switches to hi-res mode and draws a circle. A **?SYNTAX ERROR** instead means the ROM is not active; re-attach the cartridge.

5. **Load Code Probe.** Use **File -> Smart attach disk/tape image** and select **code-probe.prg**. Run with **RUN** from BASIC, or **SYS 1152**. The **:** prompt appears.

C.2 Setting Up the Super Expander via the VICE CLI

The same configuration can be expressed as a single **xvic** command, which is more reproducible than the GUI path because it does not depend on saved emulator preferences.

```
xvic -memory 3k -cartA <path-to-super-expander-rom.prg> -autostart <path-to-code-probe.prg>
```

The flags do the following:

- **-memory 3k** - enables Block 0, the same option the GUI provides under **Settings -> Machine -> Model -> RAM blocks**. No GUI tick is needed when this flag is present.
- **-cartA <path>** - attaches a generic cartridge at block 5 (**\$A000**). This handles a PRG-with-header file such as the Zimmers ROM directly; no **.crt** wrapper is required.
- **-autostart <path>** - loads the supplied PRG on boot and **RUNs** it. Code Probe ships with a small BASIC stub at **\$0401** that redirects **RUN** to **SYS 1152**, so the monitor's **:** prompt appears within a second or two of the cold-boot banner.

To confirm the cartridge is active before launching Code Probe, run **xvic** without **-autostart** once and use the **GRAPHIC 2 : CIRCLE** test described in Appendix C.1. Once verified, add the **-autostart** flag to the command line for routine launches.

The **xvic** binary lives inside the VICE installation directory: on Windows it is typically **xvic.exe** under the **bin** folder of the WinVICE install, and on Linux and macOS it is on the default **PATH** after a package-manager install.

